

Assignment # 3

23L-0515 Ahsan Baig

23L-0569 Abdullah

23L-0913 Bissam If

Q1)

Dynamic Programming can be used here to check all paths for m steps. If after m steps we arrive at the dest. we count that path. Otherwise, we ignore the path.

CountPathWithEdge($G, src, m, dest$)

{

 $V = G.V.size()$ $res[0..V][0..m]$ $res[src][0] = 1$ for each v in $G.V$

{

for $k=1$ to m

{

 $res[v][k] = 0$ for each vertex u in $G.adj[v]$

{

if (u, v) is edge in G $res[v][k] += res[u][k-1]$

}

}

return $res[dest][m]$

{

→ Time Complexity

 $O(m * |V| * |E|)$

because of nested for

Q2)

When an edge is removed from the MST, it will split into exactly 2 connected components. Since the original graph is connected, there must be an edge that exists which connects the 2 components. We just need to find the min weight edge and that including that will make up the MST.

UpdateMST($G, T, \text{weights}, a, b, n$)

```
{
    component A = BFS( $T, a, \text{excluding edge}(a, b)$ )
    component B =  $V - \text{component A}$ 
    min_weight = INT_MAX
    min-edge = NULL

    for each edge  $(u, v)$  in  $G$ 
    {
        if ( $u$  in component A &&  $v$  in component B) ||
           ( $v$  in component A &&  $u$  in component B)
        {
            if ( $\text{weights}[(u, v)] < \text{min\_weight}$ )
            {
                min_weight =  $\text{weights}[(u, v)]$ 
                min-edge =  $(u, v)$ 
            }
        }
    }
}
```

$T.\text{remove}(a, b)$

$T.\text{append}(\text{min-edge})$

return T

}

→ Time Complexity

$O(|V| + |E|)$

Finding components $O(|V|)$

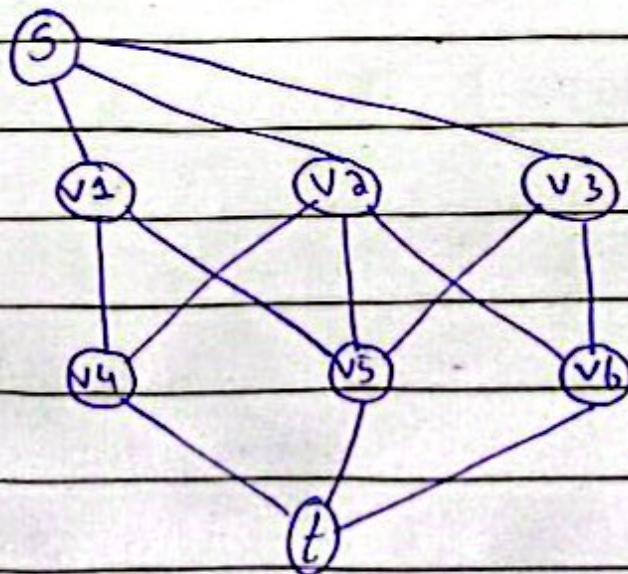
Finding min-cross

edge : $O(|E|)$

-(Q13)-

No, DFS cannot compute the number of distinct paths between two vertices in $O(|V| + |E|)$ time. The number of paths can be exponential.

Even in a small graph, the number of distinct paths can grow exponentially in size. And one cannot count exponential objects in linear time so standard DFS visit is not enough to check all paths.



Total 7 distinct paths from S to t

— of 0/14 p —

Graph will have Strongly connected components. So, find all SCCs & create a DAG. Identify sources & sinks in DAG, & connect them optimally by adding edges from sinks back to source in a cycle.

Pseudocode

Min Rods to Connect (G, n , townhall)

```
{
    scc_id[1], num_sccs = kosaraju_scc( $G, n$ )
    if (num_scc == 2)
        { return (0, 1) }
    in_degree[num_sccs] = 0
    out_degree[num_sccs] = 0
    for each ( $u, v$ ) in  $G$ 
        {
            scc_u = scc_id[u]
            scc_v = scc_id[v]
            if (scc_u  $\neq$  scc_v)
                {
                    out_degree[scc_u]++
                    in_degree[scc_v]++
                }
        }
}
```

```
sources = []  
sinks = []  
for ( i=0 to num_nodes - 1 )  
{  
    if ( in_degree[i] == 0 )  
        source.append(i)  
    if ( out_degree[i] == 0 )  
        sinks.append(i)  
}  
  
min_roads = max ( |sources|, |sinks| )  
new_edges = []  
for ( i=0 to min_roads - 1 )  
{  
    source_idx = sources [ i % |sources| ]  
    sink_idx = sinks [ i % |sinks| ]  
    u = representative [ sink_idx ]  
    v = representative [ source_idx ]  
    new_edges.append ( ( u, v ) )  
}  
return ( min_roads, new_edges )  
}
```


M T W T F S

DATE: _____

kosaraju - SCC (G, n)

{ visited = [False] * n

finish_order = []

for each vertex v in V

{ if (not visited[v])

{ DFS (v, G, visited, finish_order

}

}

G_reverse = reverse edges of G;

visited = [False] * n

scc_id = [-1] * n

current_scc = 0

for (each vertex v in reverse(finish_order))

{ if (not visited[v])

{

DFS - Assign - SCC (v, G_reverse, visited,

scc_id, current_scc)

current_scc ++

}

}

return (scc_id, current_scc)

}

Time complexity :- $O(|V| + |E|)$

— (QH5) —

Since, every path b/w all pairs of nodes has at most 8 edges, we can use a modified Bellman-Ford algorithm that runs for 8 iterations instead of $V-1$.

Pseudocode

Single source ^{shortest} path (G, s, n)

{ dist[0..n]

for (i = 0 to n-1)

{ dist[i] = INI-MAX }

for (i = 1 to 8)

{ updated = false

for (each edge (u,v,w) in E)

{ if (dist[u] + w < dist[v])

{ dist[v] = dist[u] + w

updated = true

}

}

M T W T F S

DATE: _____

if (!updated)

break

}

return dist

}

Time complexity : $O(|V| + |E|)$

$O(|V|)$ for first loop

$O(8 \times |E|)$ for second loop, which

approximates to $O(|E|)$